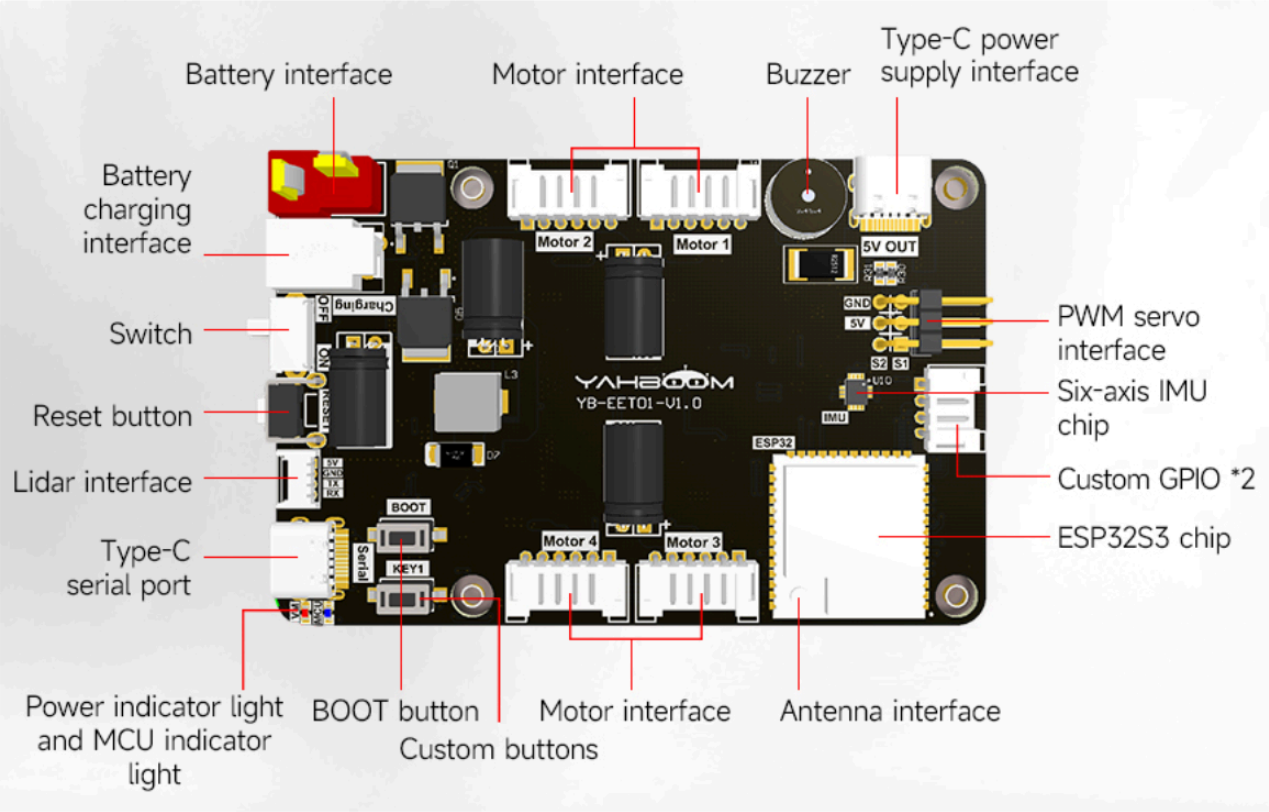
7503

Note：电控控制

电机

- **白色插头 → 控制板**
- **黑色插头 → 电机**
  Motor1 左前轮电机
  Motor2 左后轮电机
  Motor3 右前轮电机
  Motor4 右后轮电机

板子



Code框架：

```
dase7503_motor_control/
├── README.md ← 详细使用说明
├── single_motor_pwm/ ← 阶段1：单电机开环测试
│   └── single_motor_pwm.ino
├── single_motor_pid/ ← 阶段2：单电机 PID 闭环
│   └── single_motor_pid.ino
├── four_motors_pid/ ← 阶段3：四电机独立 PID
│   └── four_motors_pid.ino
└── mecanum_cmd_vel/ ← 阶段4：最终版本（Mecanum + /cmd_vel）
    └── mecanum_cmd_vel.ino
```

串口
查看串口

```
ls /dev/tty*
```

给权限

```
sudo chmod 666 /dev/ttyUSB0
sudo usermod -aG dialout $USER
```

重新登录或执行 newgrp dialout

启动 micro-ROS Agent

```
docker run -it --rm -v /dev:/dev --privileged --net=host \
  microros/micro-ros-agent:jazzy serial --dev /dev/ttyUSB0 -v6
```

单电机开环

```
ros2 topic echo /rpm
ros2 topic pub --once /set_pwm std_msgs/msg/Int32 "{data: 150}"   # 正转
ros2 topic pub --once /set_pwm std_msgs/msg/Int32 "{data: -150}"  # 反转
ros2 topic pub --once /set_pwm std_msgs/msg/Int32 "{data: 0}"     # 停
```

单电机pid

```
ros2 topic echo /rpm
ros2 topic pub --once /set_rpm std_msgs/msg/Int32 "{data: 120}"
ros2 topic pub --once /set_pid std_msgs/msg/Float32MultiArray "{data: [2.5,
0.3, 0.05]}"
```

四电机

```
ros2 topic echo /rpms
ros2 topic pub --once /set_rpms std_msgs/msg/Float32MultiArray "{data:
[100.0, 100.0, 100.0, 100.0]}"
```

最终 Mecanum + cmd_vel
暂定 PID data: [2.5, 0.3, 0.05]

```
# 前进
ros2 topic pub /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.2}}" --rate
10
```

```
# 左侧移
ros2 topic pub /cmd_vel geometry_msgs/msg/Twist "{linear: {y: 0.15}}" --rate
10

# 原地左转
ros2 topic pub /cmd_vel geometry_msgs/msg/Twist "{angular: {z: 0.8}}" --rate
10

# 停止
ros2 topic pub --once /cmd_vel geometry_msgs/msg/Twist "{}"

# 看四个轮子实际转速
ros2 topic echo /wheel_rpms
```

检查有无反转：只给前进指令
ros2 topic pub /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.15}}" --rate 10
如果某个轮子反着转 → 需要在代码里把对应电机的 setpoint 取反。
1，2反着

```
motors[0].setpoint = constrain(-v_lf * factor, -MAX_RPM, MAX_RPM); // LF
motors[1].setpoint = constrain(-v_lr * factor, -MAX_RPM, MAX_RPM); // LR
motors[2].setpoint = constrain(v_rf * factor, -MAX_RPM, MAX_RPM); // RF
motors[3].setpoint = constrain(v_rr * factor, -MAX_RPM, MAX_RPM); // RR
```

如果某个轮子反了，就在前面加负号，例如左前轮反了：

```
motors[0].setpoint = constrain(-v_lf * factor, -MAX_RPM, MAX_RPM); // 加了负
号
```

控制逻辑
/cmd_vel (x, y, z)
↓
Mecanum 逆运动学
↓
四个轮子的目标 RPM
↓
PID 速度环
↓
PWM 输出
↓
电机转动

需要测量

```
#define WHEEL_RADIUS   0.0325f   // 轮半径（米）
#define Lx             0.10f     // 左右轮中心距的一半（米）
#define Ly             0.10f     // 前后轮中心距的一半（米）
#define MAX_RPM        180.0f    // 单个电机最大允许转速（保护用）
```

推荐调参顺序（经典方法）

1. **先只调 Kp**（Ki=0, Kd=0）
   - 从小往大加。
   - 目标：反应变快，但不要剧烈震荡。
2. **再加 Ki**
   - 消除稳态误差（实际转速到不了目标值的问题）。
   - Ki 太大容易震荡或超调。
3. **最后加少量 Kd**
   - 抑制超调和震荡，让曲线更平滑。
   - Kd 通常比较小。

看曲线

```
ros2 run rqt_plot rqt_plot
```

在输入框里输入 /rpm/data 或 /wheel_rpms/data[0] 等，点 +。

- 单电机：/rpm/data
- 四电机：/rpms/data[0]、/rpms/data[1]...
- Mecanum：/wheel_rpms/data[0] 等

好的表现：

- 给目标转速后，实际转速快速上升，几乎没有或只有很小超调。
- 到达目标后基本稳住，波动小。
- 负载变化时还能较快回到目标值。

不好的表现：

- 一直震荡 → Kp 或 Ki 太大。
- 到不了目标值（有稳态误差）→ 需要加大 Ki。
- 反应太慢 → 加大 Kp。

问题	解决方法
上传失败	按住 BOOT 键再点 Upload，或检查端口权限
Agent 连不上	确认串口正确，docker 加了 --privileged

问题	解决方法
电机不转	检查白/黑插头是否接反，STB 使能是否打开
方向反了	在代码里对某个电机的 setpoint 取反
RPM 乱跳	编码器中断引脚确认，增加滤波
跑偏	四轮 PID 参数尽量一致，检查机械对称性